

Torre EV — Painel do Síndico

Gestão de recarga de veículos elétricos em condomínio

Henrique de Almeida Marangoni Inacio

Alex Junior

Gustavo Brito

André Moreira

Programação de Dispositivos Móveis

MIT App Inventor 2

14 de agosto de 2026

Resumo. O Torre EV é um painel de controle para o síndico acompanhar, em tempo real, a recarga de veículos elétricos na garagem comum de um condomínio antigo. O aplicativo soma a potência das recargas em andamento, compara com o limite elétrico do prédio, alerta por texto e por voz quando a carga se aproxima do limite, recusa recargas que o estourariam e registra o consumo em kWh por morador. Foi desenvolvido no MIT App Inventor 2 e não exige nenhuma obra na rede elétrica existente.

Sumário

1	Definição do problema	3
1.1	O cenário	3
1.2	As dores, por ator	3
1.3	O nicho escolhido e o usuário final	3
2	Visão geral da solução	4
2.1	A regra do semáforo	4
2.2	Navegação entre telas	4
2.3	Persistência: o que o TinyDB guarda	4
3	Telas da solução	5
3.1	Tela 1 — Login	5
3.2	Tela 2 — Painel	5
3.3	Tela 3 — Cadastro de recarga	7
3.4	Tela 4 — Histórico	7
4	Diagrama de blocos e algoritmo	8
4.1	Login: validação de senha	8
4.2	Painel: inicialização e atualização automática	9
4.2.1	O procedimento <code>AtualizarPainel</code>	10
4.2.2	Encerramento e cálculo de consumo	12

4.3	Cadastro: validação encadeada	13
4.4	Histórico: leitura do TinyDB	15
5	Proposta de valor	16
6	Limitações conhecidas	16

1 Definição do problema

1.1 O cenário

O Residencial Parque das Torres foi construído no início dos anos 2000. São duas torres, 120 apartamentos e 240 vagas de garagem. A rede elétrica foi dimensionada há duas décadas prevendo apenas iluminação das áreas comuns, portões eletrônicos e elevadores.

Recentemente, 15 moradores adquiriram veículos elétricos, e a expectativa é que esse número dobre nos próximos meses. A tentativa inicial de ligar carregadores portáteis nas tomadas comuns da garagem resultou em queda de disjuntores gerais, aumento desproporcional na conta de energia da área comum e atrito entre vizinhos.

O problema não é falta de tomada. É que **ninguém consegue enxergar quanto está sendo puxado da rede em cada momento**. O síndico só descobre que passou do limite quando o disjuntor cai — ou seja, depois do estrago.

1.2 As dores, por ator

Ator	Dor
Donos de veículo elétrico	Insegurança na recarga; falta de previsibilidade sobre quanto tempo o carro precisa ficar plugado; vontade de pagar exatamente pelo que consumiram.
Síndico e administração	Risco de sobrecarga no barramento principal, com potencial de incêndio ou interrupção do fornecimento aos elevadores; impossibilidade de saber quanto cada morador consumiu; mediação constante de conflitos.
Moradores sem veículo elétrico	Resistência a custear, via taxa condominial, a energia dos vizinhos; incômodo com cabos espalhados pelas áreas de circulação.

Tabela 1: Dores identificadas no estudo de caso.

1.3 O nicho escolhido e o usuário final

Seguindo a orientação do desafio de *não* tentar resolver todo o ecossistema em um único sistema, a equipe escolheu atacar as duas dores do bloco central da tabela: **risco na infraestrutura** e **gestão de custos**.

Usuário final: o síndico. Não o motorista, não o morador sem carro elétrico. É o síndico quem abre o aplicativo, cadastra as recargas em andamento e responde pela integridade da rede elétrica do prédio.

É importante registrar o que o aplicativo **não** é: ele não mede eletricidade. Não há sensor nem medidor. O síndico informa quem plugou o carro e com que potência, e dá baixa quando a recarga termina. O valor do sistema não está em medir, e sim em **somar, avisar antes do estrago e registrar o consumo** — transformando anotação solta em decisão informada.

2 Visão geral da solução

2.1 A regra do semáforo

O coração do aplicativo é uma classificação em três estados, recalculada automaticamente a cada dois segundos:

Estado	Condição	Resposta do aplicativo
Verde	total < 80% do limite	Cartão verde, sem alerta
Amarelo	total $\geq$ 80% do limite	Cartão amarelo e aviso por voz
Vermelho	total > limite	Cartão vermelho, notificação na tela e aviso por voz

Tabela 2: Regra de classificação da carga do barramento.

A faixa amarela é o ponto central da proposta. O vermelho já é tarde: corresponde ao disjuntor prestes a cair. O amarelo é o aviso que o síndico nunca teve.

2.2 Navegação entre telas

Tela	Nome no projeto	Função
1	Screen1	Login do síndico
2	Painel	Dashboard: status, carga, limite, ações
3	Cadastro	Registro de uma nova recarga
4	Historico	Recargas encerradas e consumo por morador

Tabela 3: As quatro telas do aplicativo.

O fluxo é Screen1 $\rightarrow$ Painel, e do painel partem e retornam Cadastro e Historico. O painel é o centro da aplicação.

2.3 Persistência: o que o TinyDB guarda

Todo o estado do aplicativo vive no componente BancoLocal, um TinyDB presente nas quatro telas. Ele grava pares tag–valor no próprio aparelho, e os dados sobrevivem ao fechamento do aplicativo.

Tag	Padrão	Conteúdo
senha	"1234"	Senha do síndico
limiteKw	22	Limite elétrico destinado às recargas
recargasAtivas	lista vazia	Lista de [nome, kW, início] em andamento
historico	lista vazia	Lista de [nome, kWh, horas, data] encerradas

Tabela 4: Estrutura de dados no TinyDB.

Nota sobre o limite de 22 kW. O enunciado do desafio descreve o condomínio (120 apartamentos, 240 vagas, 15 veículos elétricos) mas não informa a capacidade elétrica disponível. O valor de 22 kW é, portanto, uma **decisão de projeto** da equipe, e não um dado do estudo de caso. Como o limite é ajustável em tempo de execução pelo próprio síndico, o aplicativo se adapta a qualquer capacidade real que venha a ser medida.

Nota sobre o tipo dos dados. O horário de início é gravado em milissegundos (número), e não como objeto de data. O TinyDB persiste apenas texto, número, booleano e lista; um objeto de data é serializado por sua representação textual e volta como texto, inutilizável para cálculo. A duração é obtida por subtração de milissegundos.

3 Telas da solução

3.1 Tela 1 — Login

Protege o painel com a senha do síndico, lida do TinyDB. O campo tem a propriedade Password ativada, mascarando a digitação.

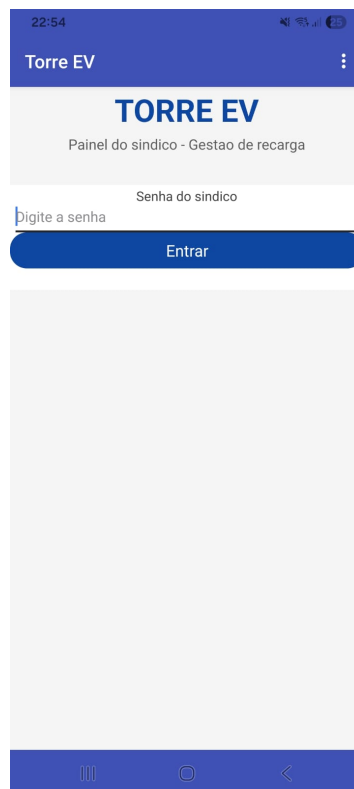


Figura 1: Tela de login. A senha padrão é 1234 e fica armazenada no TinyDB, podendo ser alterada sem recompilar o aplicativo.

3.2 Tela 2 — Painel

Concentra toda a informação de operação: estado atual, carga em kW, limite vigente, número de vagas em uso e as três ações disponíveis. Atualiza-se sozinha a cada dois segundos.

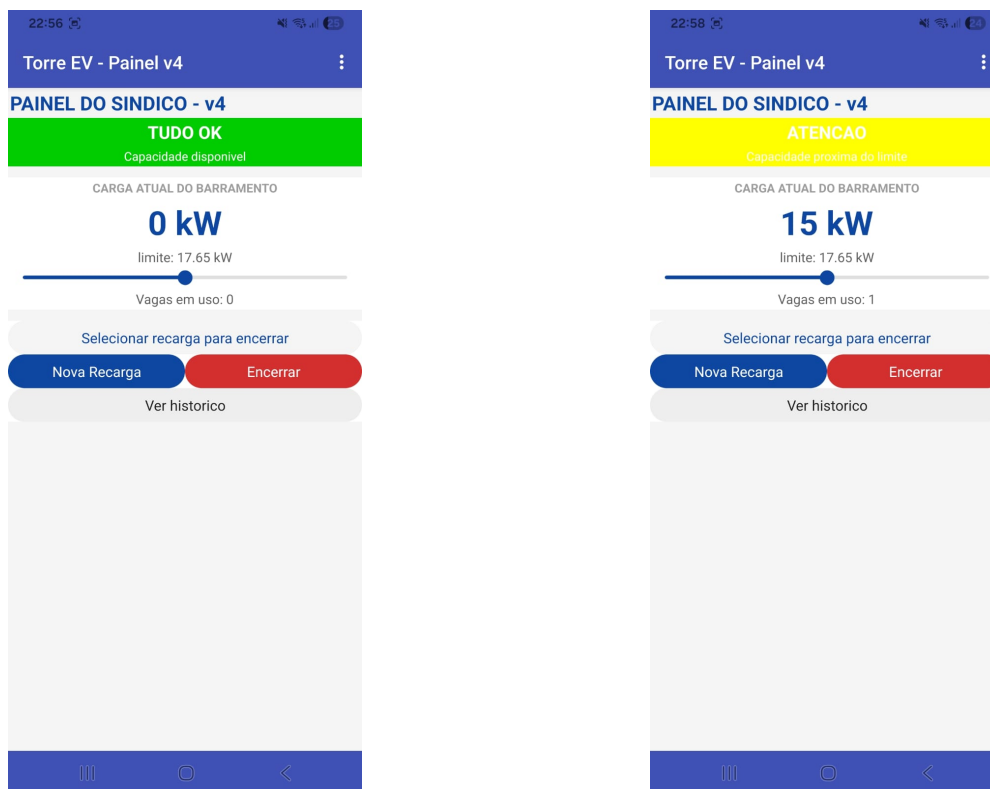


Figura 2: Painel em dois estados. À esquerda, carga folgada (verde). À direita, carga acima de 80% do limite (amarelo), situação em que o aplicativo também emite aviso por voz. A barra deslizante sob o valor de carga é o controle do limite, ajustável de 5 a 60 kW.

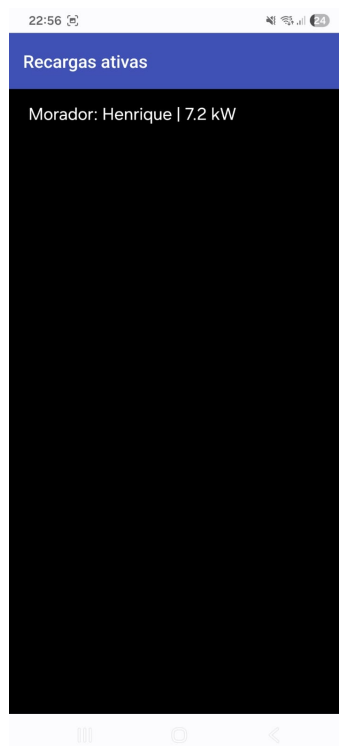


Figura 3: Seleção da recarga a encerrar. A lista é montada a partir das recargas ativas gravadas no TinyDB e mostra o morador e a potência de cada uma.

3.3 Tela 3 — Cadastro de recarga

Recebe o nome do morador e a potência do carregador. O campo de potência aceita ponto ou vírgula como separador decimal, e a validação ocorre no momento de salvar.

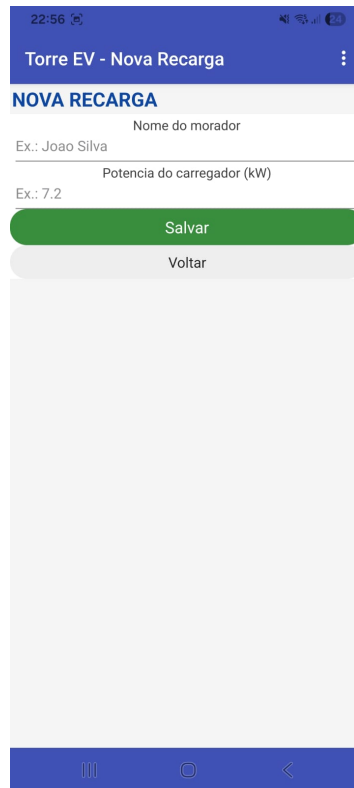


Figura 4: Tela de cadastro. Os textos de exemplo (*hints*) orientam o formato esperado em cada campo.

3.4 Tela 4 — Histórico

Lista as recargas já encerradas. Cada linha traz cinco informações: morador, energia consumida, duração, potência do carregador e o instante do encerramento.



Figura 5: Histórico de recargas encerradas. A potência exibida é derivada da razão entre energia e duração, e não de um campo adicional — decisão que mantém compatibilidade com registros gravados por versões anteriores do aplicativo.

4 Diagrama de blocos e algoritmo

Esta seção percorre a lógica do aplicativo, com ênfase nas estruturas condicionais encadeadas e no uso do TinyDB, conforme pedido no enunciado.

4.1 Login: validação de senha

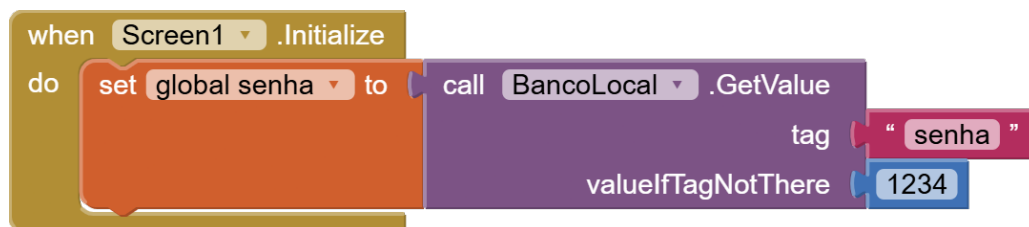


Figura 6: Screen1.Initialize. Ao abrir o aplicativo pela primeira vez, o TinyDB ainda não tem a tag `senha`; o segundo argumento de `GetValue` fornece o valor padrão 1234, que é então gravado. Nas execuções seguintes o valor lido é o que estiver armazenado.

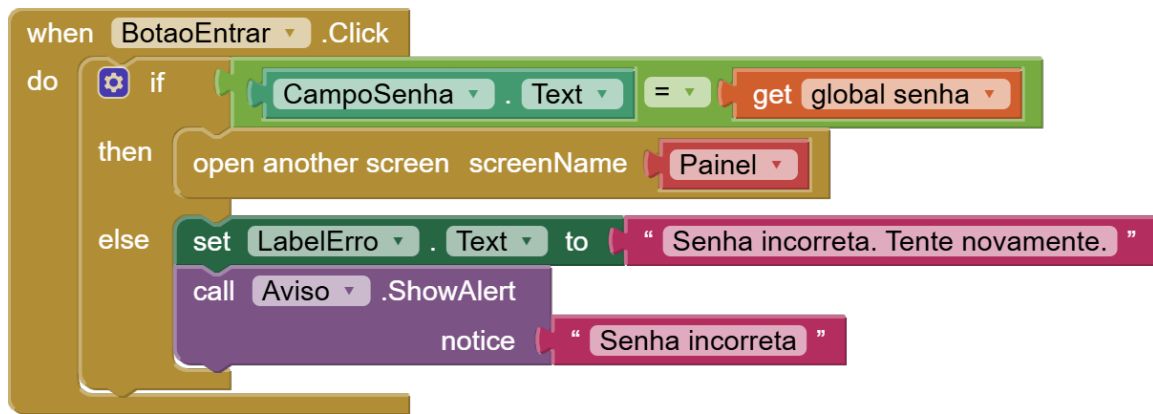


Figura 7: BotaoEntrar.Click. Condicional simples: se o texto digitado for igual à senha armazenada, abre o painel; caso contrário, escreve a mensagem de erro no rótulo LabelErro e permanece na tela.

4.2 Painel: inicialização e atualização automática

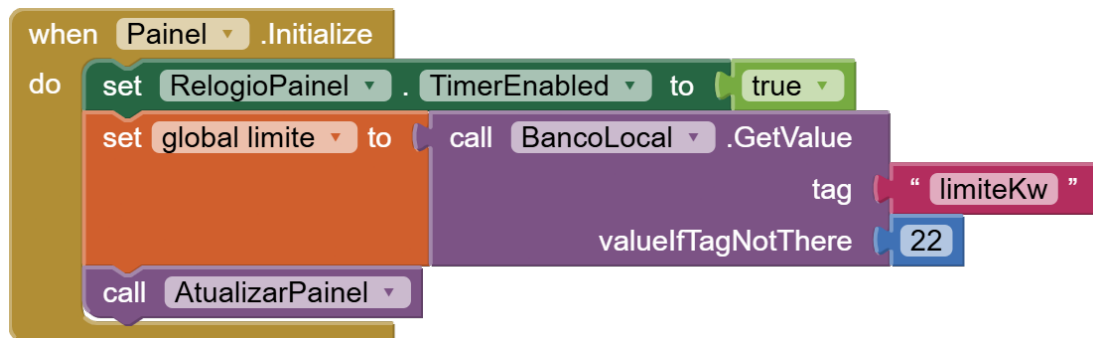


Figura 8: Painei.Initialize. Liga o temporizador, carrega o limite gravado no TinyDB (com 22 kW como padrão), posiciona a barra deslizante no valor carregado e faz a primeira atualização do painel.

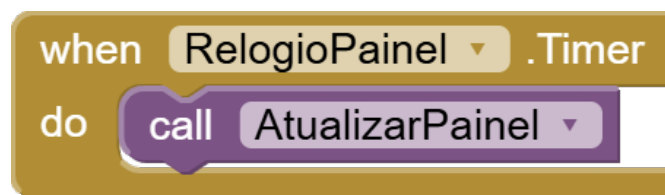


Figura 9: RelogioPainei.Timer. A cada 2000 ms o procedimento AtualizarPainei é executado. É isso que torna o painel um monitor, e não uma tela estática.

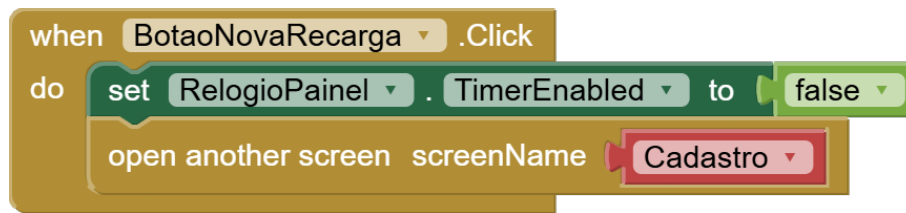


Figura 10: BotaoNovaRecarga.Click. O temporizador é **desligado** antes de abrir outra tela. Sem isso, o procedimento do painel continuaria sendo disparado enquanto outra tela estivesse em primeiro plano, tentando escrever em rótulos que não existem naquele contexto.

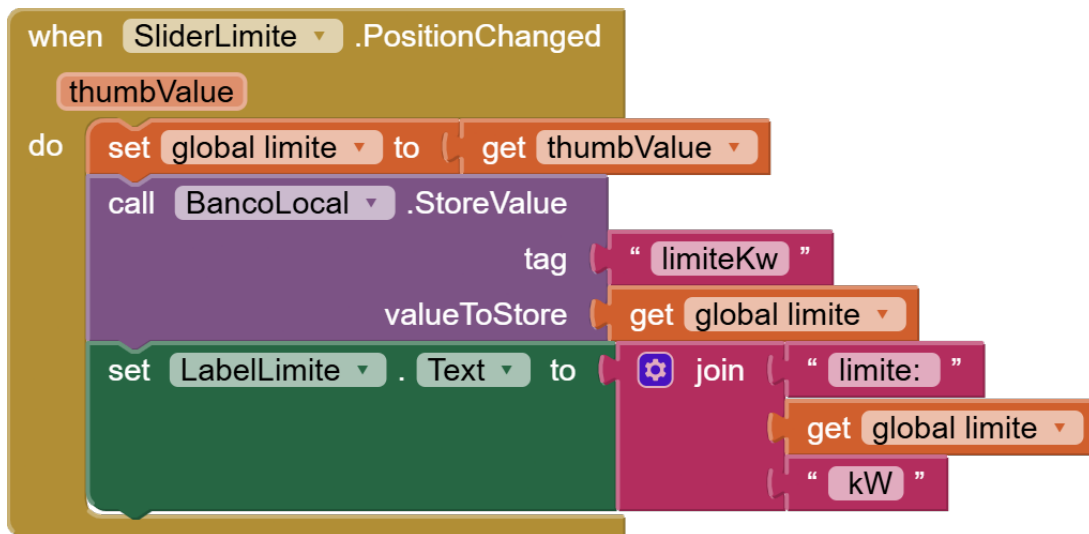


Figura 11: SliderLimite.PositionChanged. Ao arrastar a barra, o novo valor vira o limite corrente, é gravado no TinyDB e refletido no rótulo. A próxima execução de AtualizarPainel já reavalia o semáforo contra o limite novo.

4.2.1 O procedimento AtualizarPainel

É o bloco central do aplicativo (Figura 12). O algoritmo é:

1. Zera o acumulador **total** e a lista **nomes**.
2. Percorre **recargasAtivas**, lida do TinyDB, somando a potência de cada recarga e montando a descrição que aparecerá no seletor.
3. Escreve nos rótulos de carga, limite e vagas em uso.
4. Atualiza o seletor **apenas se a lista de nomes mudou**, comparando com **nomesAnt**. Sem essa comparação, a seleção feita pelo síndico seria descartada a cada dois segundos.
5. Classifica a carga em uma **condicional encadeada de três ramos**: se **total > limite**, **senão se total > 80% do limite**, **senão normal**.
6. Dentro de cada ramo, uma segunda condicional compara o estado atual com **statusAnt**. Cor e textos são reescritos, e o alerta sonoro é disparado, **apenas na transição** de estado. É isso que impede o aplicativo de repetir o mesmo aviso a cada dois segundos.

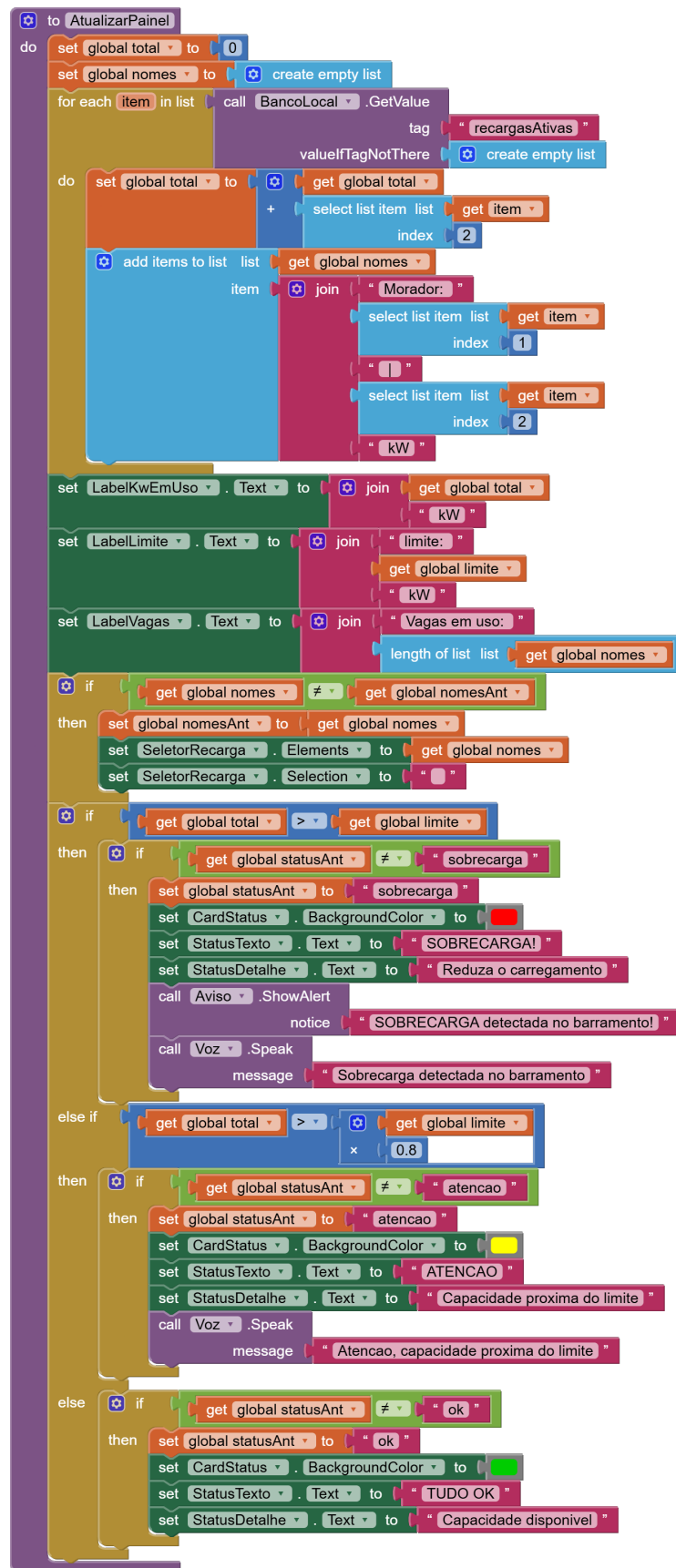


Figura 12: Procedimento AtualizarPainel, executado a cada dois segundos. A condicional externa classifica a carga; as internas garantem que o alerta ocorra somente na mudança de estado.

4.2.2 Encerramento e cálculo de consumo

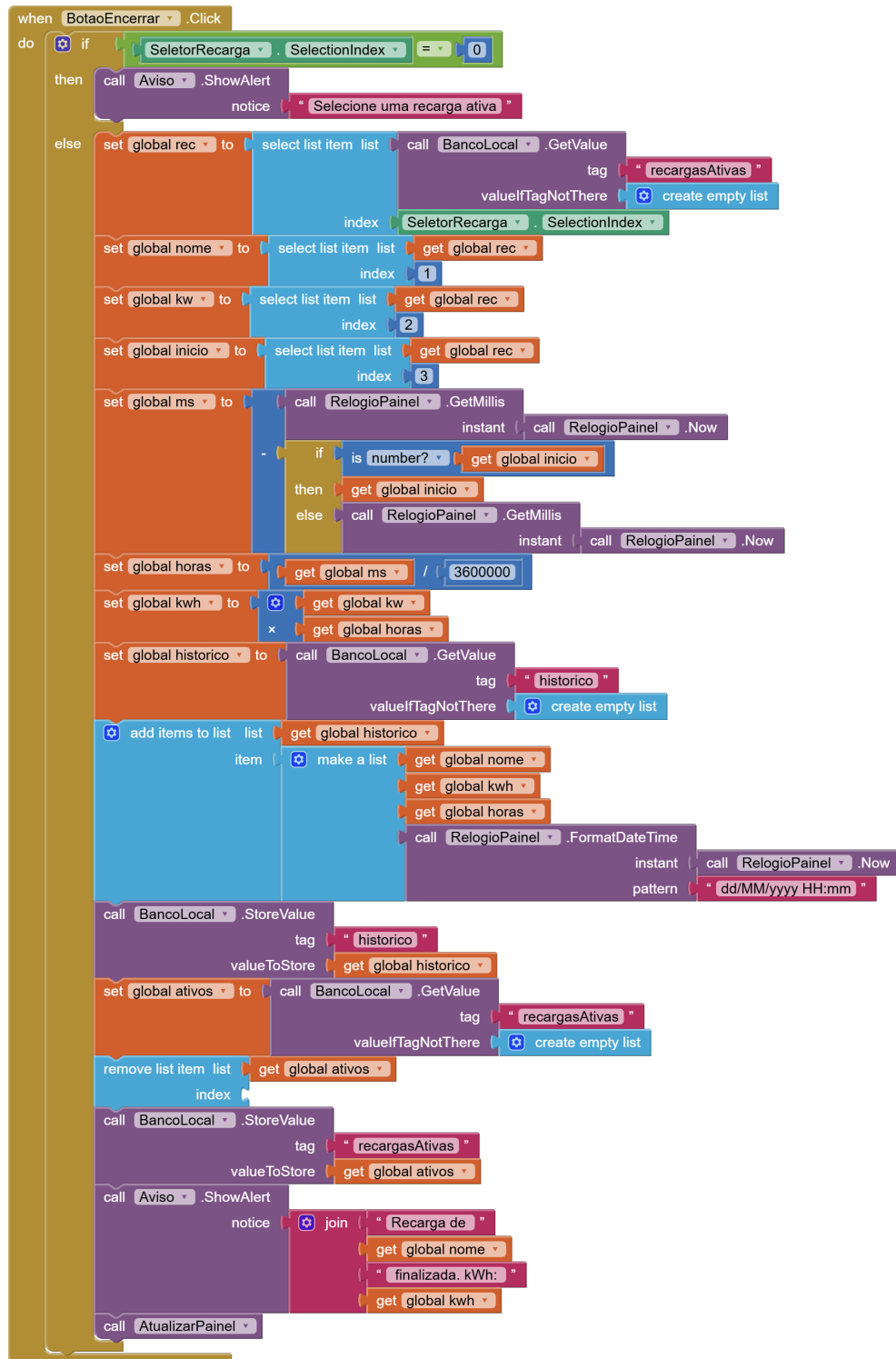


Figura 13: BotaoEncerrar.Click. Primeiro verifica se há alguma recarga selecionada. Havendo, recupera o registro da lista de ativas, calcula a duração pela diferença de milissegundos, obtém a energia consumida multiplicando potência por horas, acrescenta o resultado ao histórico no TinyDB, remove a recarga da lista de ativas, confirma ao usuário e atualiza o painel.

O cálculo, em três passos:

$$\text{ms} = \text{agora} - \text{início} \quad \text{horas} = \frac{\text{ms}}{3\,600\,000} \quad \text{kWh} = \text{kW} \times \text{horas}$$

Um carregador de 7,2 kW ligado por duas horas e meia consome, portanto, 18 kWh. A distinção entre as duas unidades é o que torna a cobrança possível: **kW** é a velocidade com que a energia é puxada, **kWh** é a quantidade efetivamente consumida. Só a segunda vira dinheiro na conta do condomínio.

4.3 Cadastro: validação encadeada

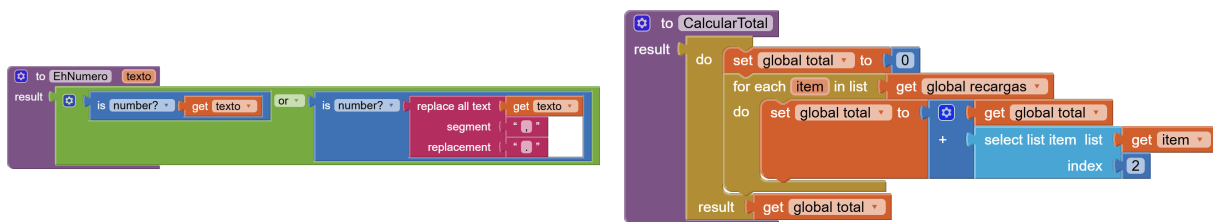


Figura 14: Os dois procedimentos auxiliares do cadastro. **EhNumero** aceita o valor tanto com ponto quanto com vírgula, testando as duas formas. **CalcularTotal** soma a potência das recargas já ativas, e é o que permite saber se ainda há capacidade disponível.

A Figura 15 mostra a validação encadeada exigida pelo enunciado. São quatro ramos, avaliados em ordem:

1. **Nome vazio** → “Informe o nome do morador”.
2. **Potência inválida** — campo vazio, não numérico, menor que 1 ou maior que o limite → mensagem dinâmica com a faixa aceita.
3. **Capacidade insuficiente** — a soma das recargas ativas com a nova ultrapassaria o limite → “Recarga não permitida”.
4. **Caso contrário** — monta a lista [nome, potência, início], grava em **recargasAtivas** no TinyDB, confirma e fecha a tela.

O terceiro ramo é o que diferencia o aplicativo de um caderno de anotações: ele **recusa** uma operação que colocaria a rede em risco, em vez de apenas registrá-la.

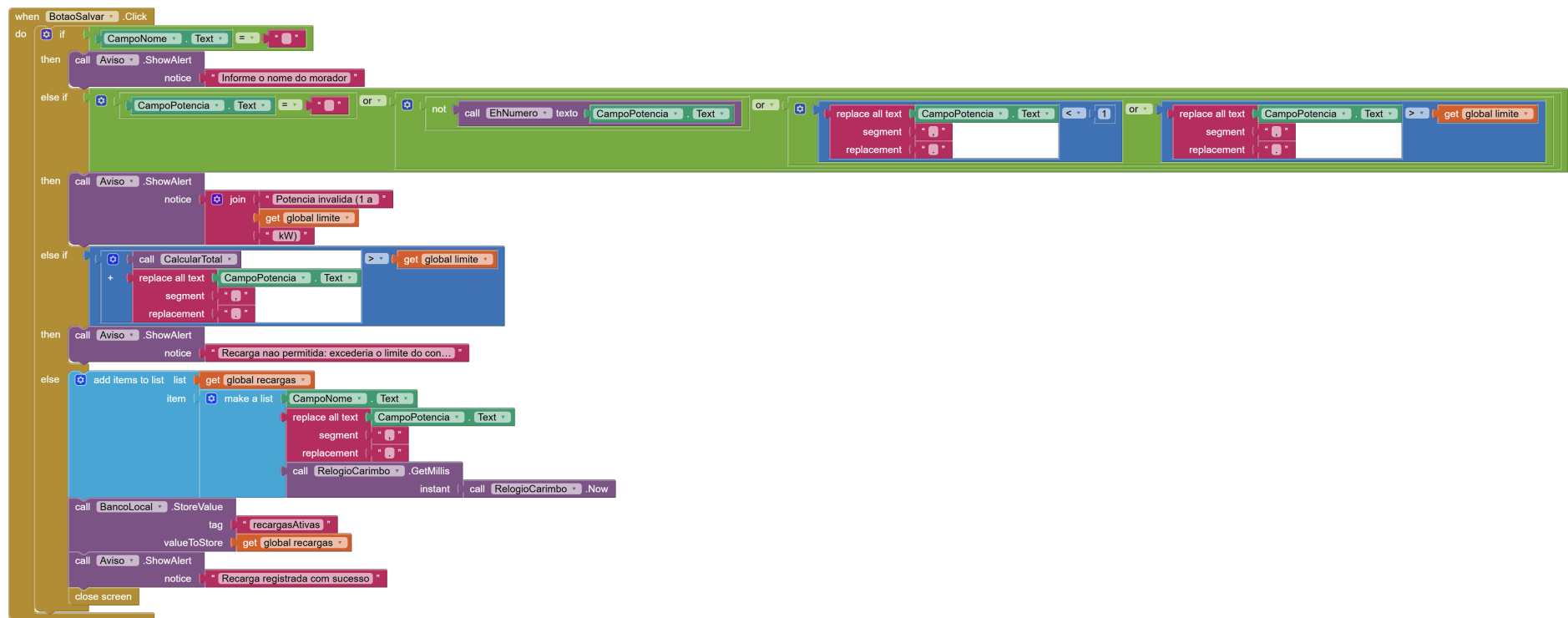


Figura 15: BotaoSalvar.Click — a validação encadeada do cadastro. Os três primeiros ramos rejeitam a entrada com mensagens específicas; o último grava a recarga no TinyDB e fecha a tela. Página em orientação paisagem por causa da largura do diagrama.

4.4 Histórico: leitura do TinyDB

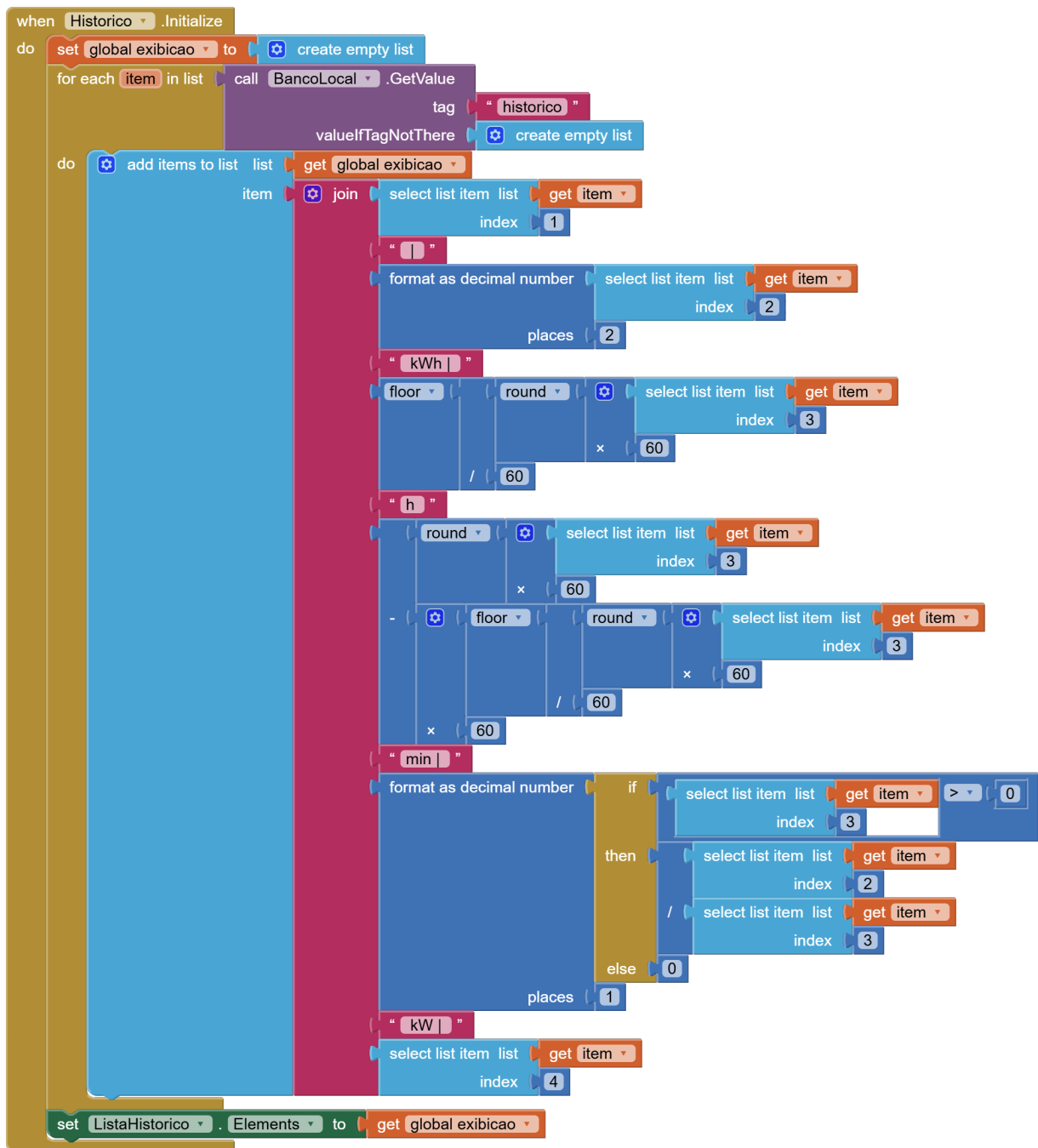


Figura 16: `Historico.Initialize`. Percorre a lista `historico` do TinyDB e monta, para cada registro, uma linha de texto formatada: energia com duas casas decimais, duração convertida de horas decimais para horas e minutos, e potência derivada da razão entre energia e duração. O resultado alimenta a `ListaHistorico`.

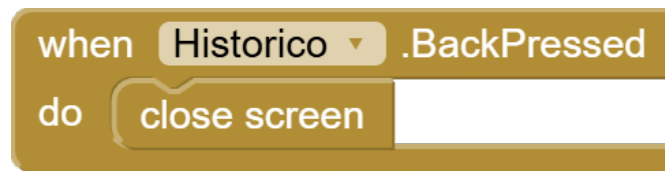


Figura 17: `Historico.BackPressed`. Trata o botão físico de voltar do aparelho, fechando a tela pelo mesmo caminho do botão “Voltar”. Sem esse tratamento, o comportamento fica a cargo do sistema operacional.

5 Proposta de valor

Para um condomínio antigo, a solução tecnicamente correta é reforçar a rede elétrica. Isso custa caro, depende de assembleia, de orçamento e de rateio — e os moradores sem veículo elétrico têm motivo para votar contra.

O Torre EV não substitui essa obra. Ele resolve outro problema, que existe **hoje** e não tem solução de custo zero: dar ao síndico a informação que falta para conviver com o limite atual sem derrubar o prédio.

Antes	Depois
Ninguém sabe quanto está sendo puxado da rede	Painel mostra a carga total em tempo real
Descobre-se o excesso quando o disjuntor cai	Aviso sonoro aos 80% do limite, antes do estrago
Recarga que estoura a rede é ligada assim mesmo	O cadastro é recusado com a justificativa
Consumo diluído na conta da área comum	kWh registrado no nome de cada morador

Tabela 5: O que muda com a adoção do aplicativo.

Três características sustentam a adequação ao contexto de um prédio antigo:

- **Custo zero de infraestrutura.** Roda no celular que o síndico já tem. Não exige sensor, medidor, obra ou aprovação em assembleia.
- **Limite configurável.** A capacidade disponível varia de prédio para prédio e pode mudar após uma reforma. O síndico ajusta na própria tela, sem alterar o aplicativo.
- **Dados locais.** Nenhuma dependência de internet, servidor ou conta em serviço externo — o que também elimina qualquer questão sobre dados pessoais dos moradores saindo do prédio.

6 Limitações conhecidas

Registrar os limites faz parte da honestidade técnica do trabalho:

- **O aplicativo não mede energia.** Os dados são informados pelo síndico. Se ele esquecer de dar baixa, o painel continuará considerando a recarga ativa.

- **Não atua sobre o carregador.** Alerta e recusa cadastro, mas não interrompe fisicamente nenhuma recarga.
- **Uso individual.** Os dados ficam no aparelho do síndico. Troca de celular ou alternância entre dois síndicos exigiria sincronização, que está fora do escopo.
- **Não emite cobrança.** O sistema apura o consumo por morador; converter isso em rateio continua sendo trabalho da administração.
- **O histórico só recebe registro no encerramento.** Salvar uma recarga nova a coloca entre as ativas; ela só aparece no histórico depois de encerrada.